Software Architecture and Design Specification

Project: **ChatSphere : A Real-Time Chat Application**

Version: 1.0

Authors: Neelambika,Hema shushmitha n, Monish Goel , Kuber Vijayakar

Date: 28-09-2026

Status: Draft

Revision History

Version

Date

28-09-2026

Author

Neelambika

Hema
sushmitha n,

Monish Goel,

Kuber
Vijayakar

Change Summary

Initial SAD document

Approvals

Role	Name	Signature/Date
Project Guide	Vadiraja A	28-9-2026
Team Lead	Monish Goel	28-9-2026

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the architecture and design of **ChatSphere**, a real-time chat application that enables users to communicate through instant text messages.

The document describes the system architecture, major components, communication between components, database design, APIs, security mechanisms, error handling, and important design decisions.

1.2 Scope

ChatSphere provides the following major functionalities:

- User registration and login
- User authentication
- One-to-one messaging
- Real-time message delivery
- Group chat
- Online/offline status
- Message history
- Message timestamps
- Basic profile management
- Secure communication between clients and the server

The system is designed as a web-based application where users can communicate through a browser.

1.3 Audience

This document is intended for:

- Developers
- QA engineers
- Project guides/instructors
- System designers
- Security reviewers

- Maintenance teams

1.4 Definitions

Term	Meaning
ChatSphere	Name of the real-time chat application
User	Person registered in the ChatSphere system
Chat	Conversation between users
Message	Text sent by one user to another user/group
WebSocket	Protocol used for real-time bidirectional communication
REST API	HTTP-based interface used for application operations
JWT	JSON Web Token used for authentication
DB	Database used to store application data
UI	User Interface

2. Document Overview

2.1 How to use this document

This document provides the architectural and design specifications for ChatSphere.

It contains:

- Architecture goals and constraints
- Stakeholders and concerns
- UML component diagram
- Component descriptions
- Architecture pattern
- Technology stack

- Risks and mitigations
- Requirement traceability
- Security architecture
- Sequence diagrams
- API design
- Error handling
- UX considerations

2.2 Related Documents
SThe following documents are related to this SAD:

- Software Requirements Specification (SRS)
- Software Test Plan (STP)
- Requirements Traceability Matrix (RTM)
- UML diagrams

3. Architecture

3.1 Goals & Constraints

Goals

The architecture of ChatSphere aims to provide:

- Real-time message delivery
- Secure user authentication
- Reliable communication
- Scalable application structure
- Easy maintenance
- Separation of concerns
- Persistent message storage

- Responsive user experience

Constraints

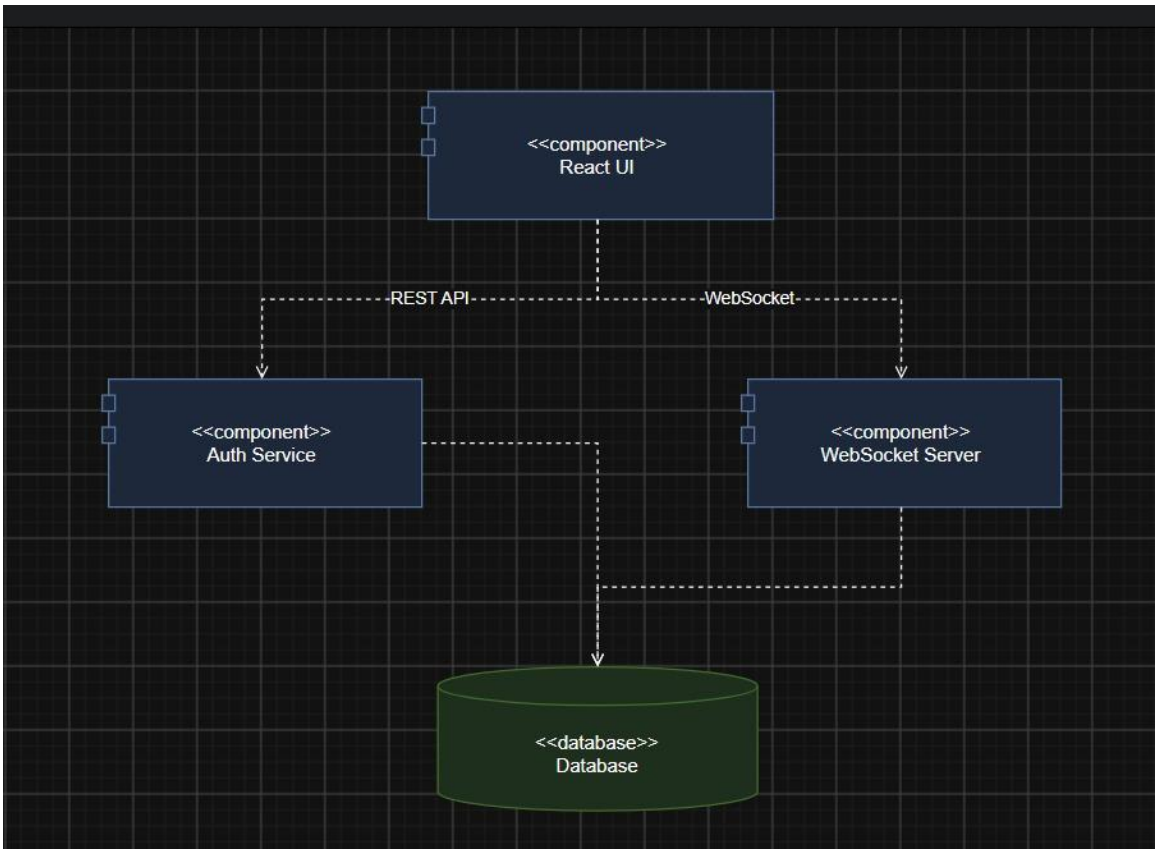
The system has the following constraints:

- Internet connection is required.
- Real-time communication depends on WebSocket connectivity.
- User credentials and messages must be protected.
- The system should work on modern web browsers.
- Database availability is required for persistent message storage.

3.2 Stakeholders & Concerns

Stakeholder	Main Concerns
Users	Fast messaging, usability, privacy
Developers	Modularity, maintainability
QA Team	Reliability and correctness
Administrator	User management and system monitoring
Project Guide	Proper architecture and implementation
Security Reviewer	Authentication and data protection

3.3 Component (UML) Diagram



3.4 Component Descriptions

1. ChatSphere Web Client

Responsible for the application's user interface.

It handles:

- Login and registration screens
- Chat list
- Message display
- Message input
- User profile
- Online/offline indicators

2. Authentication Service

Responsible for authenticating users.

It handles:

- Login
- Registration
- Password verification
- JWT generation
- Authentication validation

3. User Service

Manages user-related information.

It handles:

- User profiles
- User search
- Online/offline status
- Profile updates

4. Chat Service

Responsible for chat-related operations.

It handles:

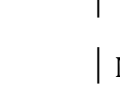
- Creating conversations
- Retrieving conversations
- Sending messages
- Retrieving message history
- Group chat management

5. Real-Time Communication Service

Responsible for real-time message delivery using WebSockets.

For example:

User A



Message

▼

WebSocket Server

│

│ Real-time event

▼

User B

This allows User B to receive a message without refreshing the webpage.

6. Database

Stores persistent application information such as:

- User accounts
- User profiles
- Conversations
- Messages
- Timestamps
- Group information

3.5 Chosen Architecture Pattern and Rationale

Chosen Pattern: Layered Architecture

ChatSphere uses a **Layered Architecture**.

The system can be divided into:

Presentation Layer React Web Application
API / Communication REST + WebSocket
Business Logic Layer Authentication + Chat
Data Access Layer Database Operations
Database

Why Layered Architecture?

It provides:

- Separation of responsibilities
- Easier debugging
- Easier testing
- Better maintainability
- Independent modification of layers
- Clear organization of application logic

A microservices architecture is not necessary for the initial version because ChatSphere can be implemented effectively as a modular application with clearly separated services.

3.6 Technology Stack & Data Stores

You can use the following stack for your project:

Layer	Technology
Frontend	React.js
Backend	Node.js + Express.js
Real-time communication	Socket.IO / WebSocket
Authentication	JWT
Database	MongoDB
API	REST API
Styling	HTML + CSS
Version Control	Git + GitHub

Database Collections

MongoDB can contain collections such as:

users

↓

conversations

↓

messages

Example message:

```
{
 "senderId": "user123",
 "conversationId": "chat456",
 "content": "Hello!",
 "timestamp": "2026-09-28T10:30:00"
}
```

3.7 Risks & Mitigations

Risk	Mitigation
WebSocket connection failure	Automatic reconnection
Unauthorized access	JWT authentication
Password theft	Password hashing
Message interception	HTTPS/WSS
Database failure	Database backup
Server overload	Connection and request management
Invalid user input	Input validation
Session/token misuse	Token expiration

3.8 Traceability to Requirements

Requirement	Architecture Component
R1 – User Registration	Authentication Service
R2 – User Login	Authentication Service
R3 – User Authentication	JWT Authentication

R4 – Send Message Chat Service

R5 – Real-Time Message Delivery WebSocket Service

R6 – Message History Database + Chat Service

R7 – User Profile User Service

R8 – Group Chat Chat Service

R9 – Online Status WebSocket Service

R10 – Secure Communication HTTPS/WSS + Authentication

3.9 Security Architecture

ChatSphere must protect user accounts and messages.

Authentication

Users authenticate using:

Username/Email + Password

↓

Authentication Service

↓

JWT Token

↓

Authenticated User

Password Security

Passwords should not be stored as plain text.

Instead:

Password

↓

Hashing Algorithm

↓

Password Hash

↓

Database

Communication Security

HTTPS should be used for REST API communication.

For real-time communication:

Client ←── Secure WebSocket (WSS) ──→ Server

Threat Considerations

Threat	**Mitigation**
Spoofing	JWT authentication
Tampering	HTTPS/WSS
Information disclosure	Encryption
Denial of Service	Rate limiting
Unauthorized access	Authentication + authorization
Malicious input	Input validation

4. Design

4.1 Design Overview

ChatSphere follows a layered design in which the frontend communicates with the backend through REST APIs and WebSocket connections.

User

↓

React UI

↓

REST API ──────────────────→ Backend

↓ ↓

WebSocket ←———————————— Chat Service

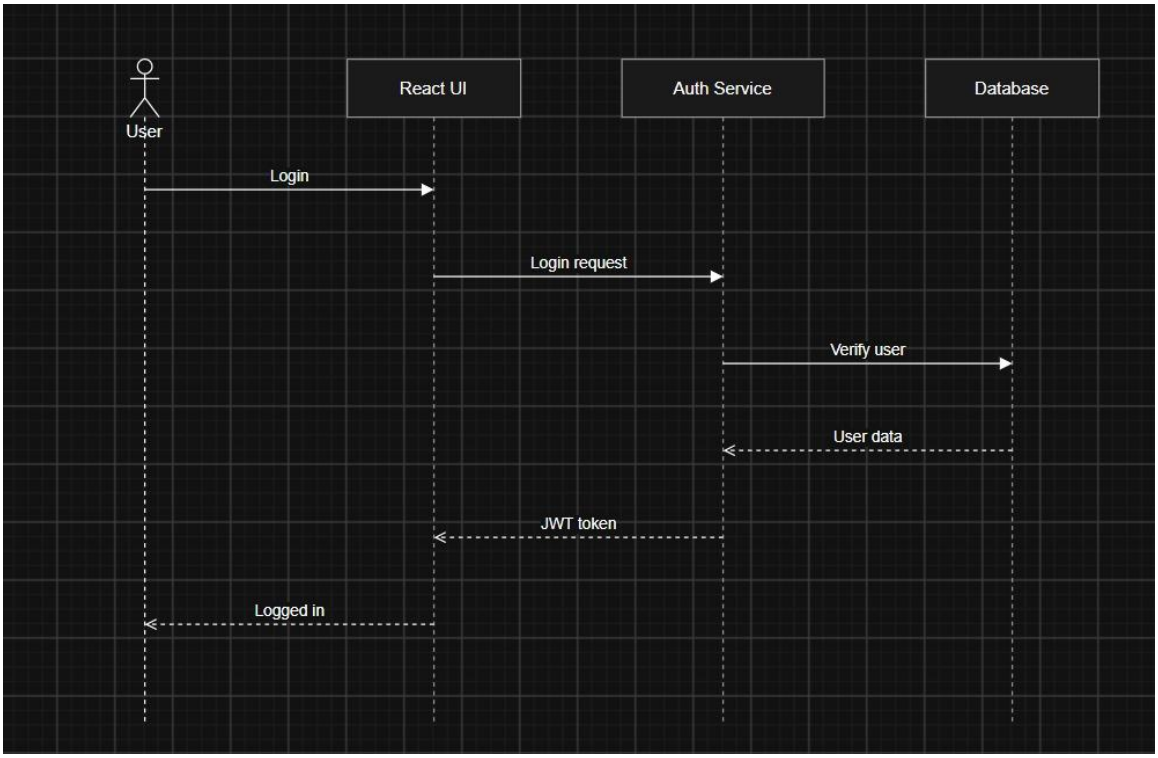
↓

MongoDB

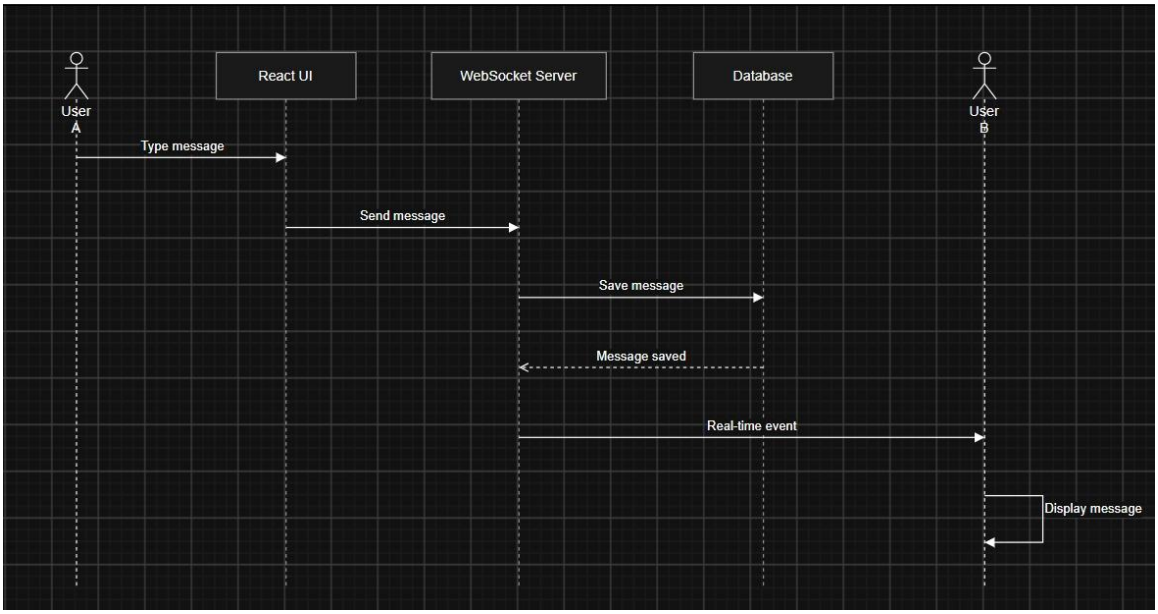
REST APIs are primarily used for operations such as authentication and retrieving data, while WebSockets are used for real-time communication.

4.2 UML Sequence Diagrams

user login



sending a message



4.3 API Design

The template specifies interface definitions for at least two components.

API 1 – Login

Endpoint

POST /api/auth/login

Request

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response

```
{
  "status": "success",
  "token": "jwt-token",
  "userId": "12345"
}
```

Errors

400 Invalid request

401 Invalid credentials

500 Internal server error

API 2 – Send Message

Endpoint

POST /api/messages

Request

```
{
  "conversationId": "chat123",
  "content": "Hello!"
```

}

Response

{

"status": "success",

"messageId": "msg456",

"timestamp": "2026-09-28T10:30:00"

}

Errors

400 Invalid message

401 Unauthorized

404 Conversation not found

500 Internal server error

WebSocket Interface

For real-time communication:

connect

↓

authenticate

↓

join_chat

↓

send_message

↓

receive_message

Example event:

send_message

{

conversationId,

content

}

Server broadcasts:

receive_message

{

senderId,

conversationId,

content,

timestamp

}

4.4 Error Handling, Logging & Monitoring

The system should use standardized error responses.

Example:

{

"status": "error",

"message": "Conversation not found"

}

Common Errors

Error	**Response**
Invalid login	401 Unauthorized
Invalid input	400 Bad Request
Conversation not found	404 Not Found
Server failure	500 Internal Server Error

Logging

The system should log:

- Login attempts
- Authentication failures
- Server errors
- WebSocket connection failures
- Database errors

Sensitive information such as passwords and authentication tokens should not be stored in logs.

Monitoring

Important metrics include:

- Active users
- WebSocket connections
- Message delivery failures
- API response time
- Server errors
- Database errors

4.5 UX Design

ChatSphere should provide a simple and intuitive interface.

Main Screens

Login

↓

Home / Chat Dashboard

↓

Chat List ——→ Conversation

| |

```
│           ├── Messages
│           ├── Message Input
│           └── Send Button
│
└── User Profile
```

Chat Interface

The chat screen should contain:

- User/group name
- Online/offline indicator
- Message history
- Message timestamps
- Text input
- Send button

The interface should also provide clear feedback when:

- A message is sent
- A message fails
- A user is offline
- The server connection is lost

4.6 Open Issues & Next Steps

Possible future enhancements include:

- Image and file sharing
- Voice messages
- Video calling
- Message reactions
- Message editing and deletion
- Read receipts

- Typing indicators
- Push notifications
- End-to-end encryption
- Message search
- Dark mode
- Mobile application

5. Appendices

5.1 Glossary: ATM, PIN, PCI-DSS, TLS, ADR.

Term	Definition
ChatSphere	->Real-time chat application
REST	->Representational State Transfer
API	-> Application Programming Interface
JWT	->JSON Web Token
WebSocket	->Real-time bidirectional communication protocol
UI	->User Interface
DB	->Database
WSS	->Secure WebSocket
HTTPS	->Secure HTTP
CRUD	->Create, Read, Update, Delete

5.2 References: IEEE 42010, OWASP, NIST SP 800-160.

For the submission, you can list:

- Software Requirements Specification (SRS)
- Software Test Plan (STP)

- Requirements Traceability Matrix (RTM)
- UML documentation
- OWASP security guidelines

5.3 Tools: PlantUML, draw.io, Swagger.

Tool	**Purpose**
Draw.io	UML diagrams
PlantUML	UML diagram generation
VS Code	Development
Git	Version control
GitHub	Source code management
Postman	API testing
MongoDB Compass	Database management
Browser Developer Tools	Frontend debugging